



Finding Similar Items

(v1.00)

Week 10: November 11, 2025

Jimmy Lin
David R. Cheriton School of Computer Science
University of Waterloo

These slides are available at <http://lintool.github.io/bigdata-2025f/>

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International
See <https://creativecommons.org/licenses/by-nc-sa/4.0/> for details



Key Questions

What do hash collisions have to do with finding similar items?

When do you use minhash? And random projections?

What are the knobs you control and what do they do?

Theme: Similarity

How similar are two items? How “close” are two items?

Equivalent formulations: large distance = low similarity

Lots of applications!

Problem: find similar items

Offline variant: extract all similar pairs of objects from a large collection

Online variant: is this object similar to something I’ve seen before?

Today!

Problem: arrange similar items into clusters

Offline variant: entire static collection available at once

Online variant: objects incrementally available

Next time!

Literature Note

Many communities have tackled similar problems:

Theoretical computer science

Information retrieval

Data mining

Databases

...

Issues

Slightly different terminology

Results not easy to compare

Applications

Near-duplicate detection

Web pages, training data, etc.

Recommendation

Similar users, similar items, similar tastes...

LSH: Four Steps

Specify distance metric
Jaccard, Euclidean, cosine, etc.

Compute representation
Shingling, tf.idf, etc.

Hash
Minhash, random projections, etc.

Examine collisions
Process items in same “bucket”

Distance Metrics

1. Non-negativity:

$$d(x, y) \geq 0$$

2. Identity:

$$d(x, y) = 0 \iff x = y$$

3. Symmetry:

$$d(x, y) = d(y, x)$$

4. Triangle Inequality

$$d(x, y) \leq d(x, z) + d(z, y)$$

Distance: Jaccard

Given two sets A, B

Jaccard similarity:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

$$d(A, B) = 1 - J(A, B)$$

Distance: Norms

Given
$$\begin{aligned} x &= [x_1, x_2, \dots, x_n] \\ y &= [y_1, y_2, \dots, y_n] \end{aligned}$$

Euclidean distance (L_2 -norm)
$$d(x, y) = \sqrt{\sum_{i=0}^n (x_i - y_i)^2}$$

Manhattan distance (L_1 -norm)
$$d(x, y) = \sum_{i=0}^n |x_i - y_i|$$

L_r -norm
$$d(x, y) = \left[\sum_{i=0}^n |x_i - y_i|^r \right]^{1/r}$$

Distance: Cosine

$$\text{Given } \begin{array}{l} x = [x_1, x_2, \dots, x_n] \\ y = [y_1, y_2, \dots, y_n] \end{array}$$

Idea: measure distance between the vectors

$$\cos \theta = \frac{x \cdot y}{|x||y|}$$

Thus:

$$\text{sim}(x, y) = \frac{\sum_{i=0}^n x_i y_i}{\sqrt{\sum_{i=0}^n x_i^2} \sqrt{\sum_{i=0}^n y_i^2}}$$

$$d(x, y) = 1 - \text{sim}(x, y)$$

Distance: Hamming

Given two bit vectors

Hamming distance: number of elements which differ

Representations

(Text)

Unigrams (i.e., words)

Shingles = n -grams

At the word level

At the character level

Feature weights

boolean

tf.idf

BM25

...

Representations

(Beyond Text)

For recommender systems:

Items as features for users

Users as features for items

For graphs:

Adjacency lists as features for vertices

For log data:

Behaviors (clicks) as features

Hash + Examine Collisions

Bad collisions

For “normal” hashing, we want to minimize collisions

Good collisions

Hash values of identical items collide – good for deduplication

Can we engineer the hash function so that *similar* items collide (on purpose)?

Collisions = hash similar items into the same “bucket”

This is Locality-Sensitive Hashing (LSH)!

Minhash

Near-Duplicate Detection of Webpages

What's the source of the problem?

Mirror pages (legit)

Spam farms (non-legit)

Additional complications (e.g., nav bars)

Relevance for
LLM training?

Naïve algorithm:

Compute cryptographic hash for webpage (e.g., MD5)

Insert hash values into a big hash table

Compute hash for new webpage: collision implies duplicate

What's the issue?

Intuition:

Hash function needs to be tolerant of minor differences

High similarity implies higher probability of hash collision

Minhash

Naïve approach: N^2 comparisons: Can we do better?

Seminal algorithm for near-duplicate detection of webpages

Used by AltaVista (?)

Setup:

Documents (HTML pages) represented by shingles (n-grams)

Jaccard similarity: dups are pairs with high similarity

Preliminaries: Representation

Sets:

$$A = \{e_1, e_3, e_7\}$$

$$B = \{e_3, e_5, e_7\}$$

Can be equivalently expressed as matrices:

Element	A	B
e_1	1	0
e_2	0	0
e_3	1	1
e_4	0	0
e_5	0	1
e_6	0	0
e_7	1	1

Preliminaries: Jaccard

Element	A	B
e_1	1	0
e_2	0	0
e_3	1	1
e_4	0	0
e_5	0	1
e_6	0	0
e_7	1	1

Let:

M_{00} = # rows where both elements are 0

M_{11} = # rows where both elements are 1

M_{01} = # rows where A=0, B=1

M_{10} = # rows where A=1, B=0

$$J(A, B) = \frac{M_{11}}{M_{01} + M_{10} + M_{11}}$$

Minhash

Computing minhash

Start with the matrix representation of the set

Randomly permute the rows of the matrix

minhash is the first row with a “one”

Example

$$h(A) = e_3 \quad h(B) = e_5$$

Element	A	B
e_1	1	0
e_2	0	0
e_3	1	1
e_4	0	0
e_5	0	1
e_6	0	0
e_7	1	1

Element	A	B
e_6	0	0
e_2	0	0
e_5	0	1
e_3	1	1
e_7	1	1
e_4	0	0
e_1	1	0

Minhash and Jaccard

Element	A	B	
e_6	0	0	M_{00}
e_2	0	0	M_{00}
e_5	0	1	M_{01}
e_3	1	1	M_{11}
e_7	1	1	M_{11}
e_4	0	0	M_{00}
e_1	1	0	M_{10}

$$P[h(A) = h(B)] = J(A, B)$$

$$\frac{M_{11}}{M_{01} + M_{10} + M_{11}} \quad \frac{M_{11}}{M_{01} + M_{10} + M_{11}}$$

Woah!

To Permute or Not to Permute?

Problem: Permutations are expensive

Solution: Interpret the hash value as the permutation

Only need to keep track of the minimum hash value

Easy to keep track of multiple minhash values at once

Extracting Similar Pairs

Task: discover all pairs with similarity greater than s

Naïve approach: N^2 comparisons: Can we do better?

Tradeoffs:

False positives: discovered pairs that have similarity less than s

False negatives: pairs with similarity greater than s not discovered

The errors (and costs) are asymmetric!

Extracting Similar Pairs

We know: $P[h(A) = h(B)] = J(A, B)$

Task: discover all pairs with similarity greater than s

Algorithm:

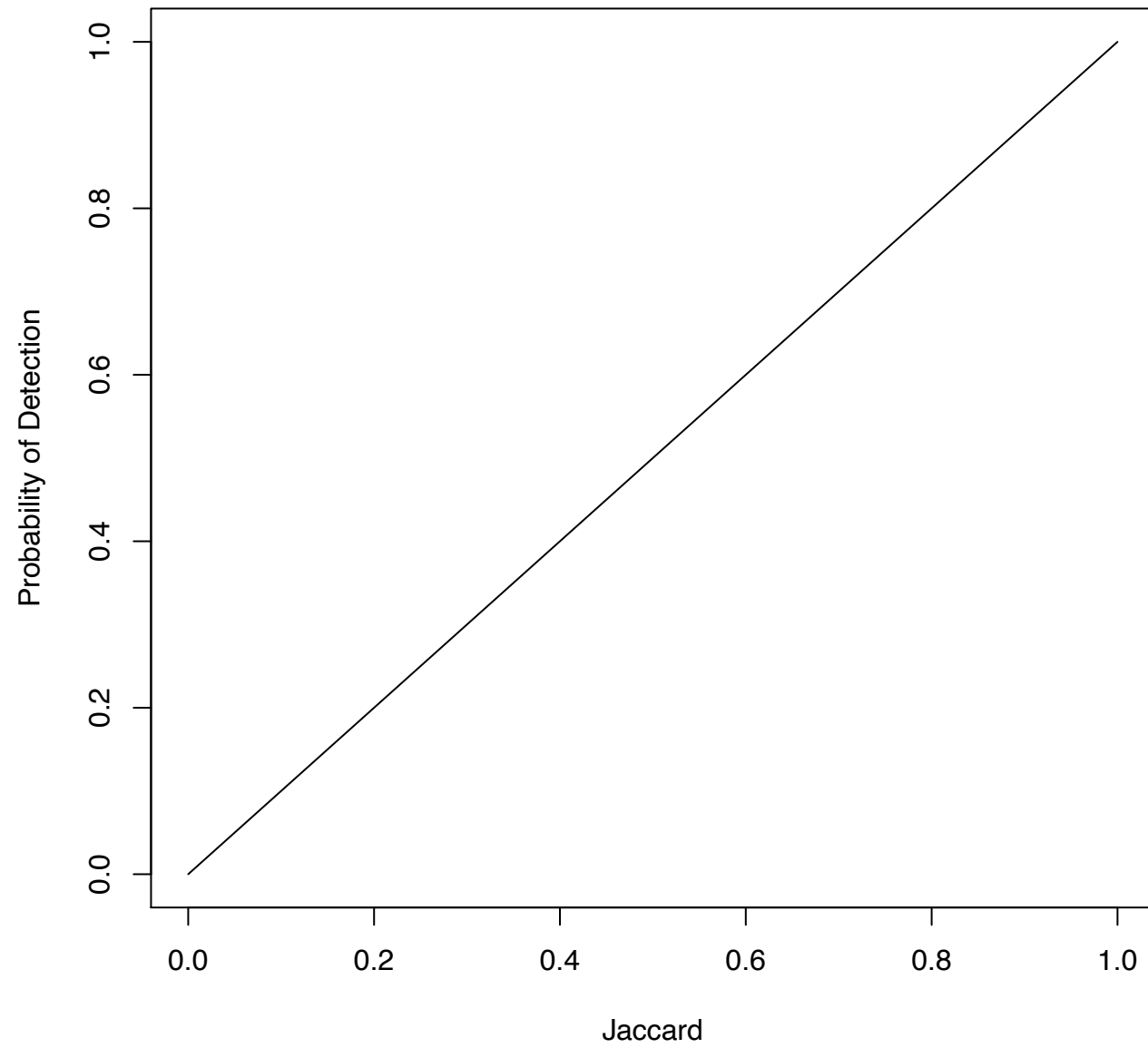
For each object, compute its minhash value

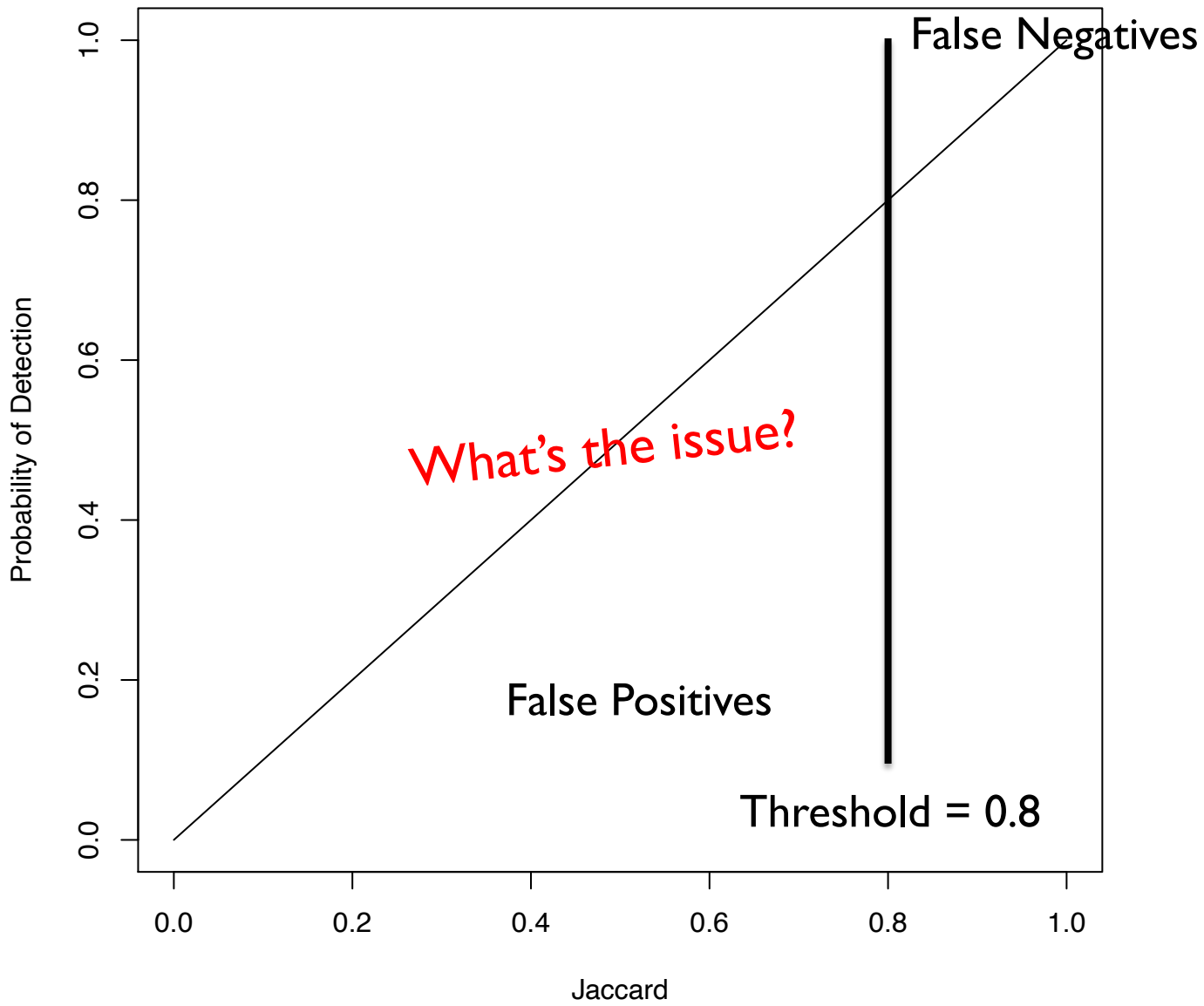
Group objects by their hash values

Output all pairs within each group

Analysis:

If $J(A, B) = s$, then probability we detect it is s
(that is, the minhash collides)





2 Minhash Signatures

We know: $P[h(A) = h(B)] = J(A, B)$

Task: discover all pairs with similarity greater than s

Algorithm:

For each object, compute 2 minhash values and concatenate = signature

Group objects by their signatures

Output all pairs within each group

Analysis:

If $J(A, B) = s$, then probability we detect it is s^2
(that is, minhash-1 collides *and* minhash-2 collides)

3 Minhash Signatures

We know: $P[h(A) = h(B)] = J(A, B)$

Task: discover all pairs with similarity greater than s

Algorithm:

For each object, compute 3 minhash values and concatenate = signature

Group objects by their signatures

Output all pairs within each group

Analysis:

If $J(A, B) = s$, then probability we detect it is s^3
(that is, minhash- $\{1, 2, 3\}$ all collide)

k Minhash Signatures

We know: $P[h(A) = h(B)] = J(A, B)$

Task: discover all pairs with similarity greater than s

Algorithm:

For each object, compute k minhash values and concatenate = signature

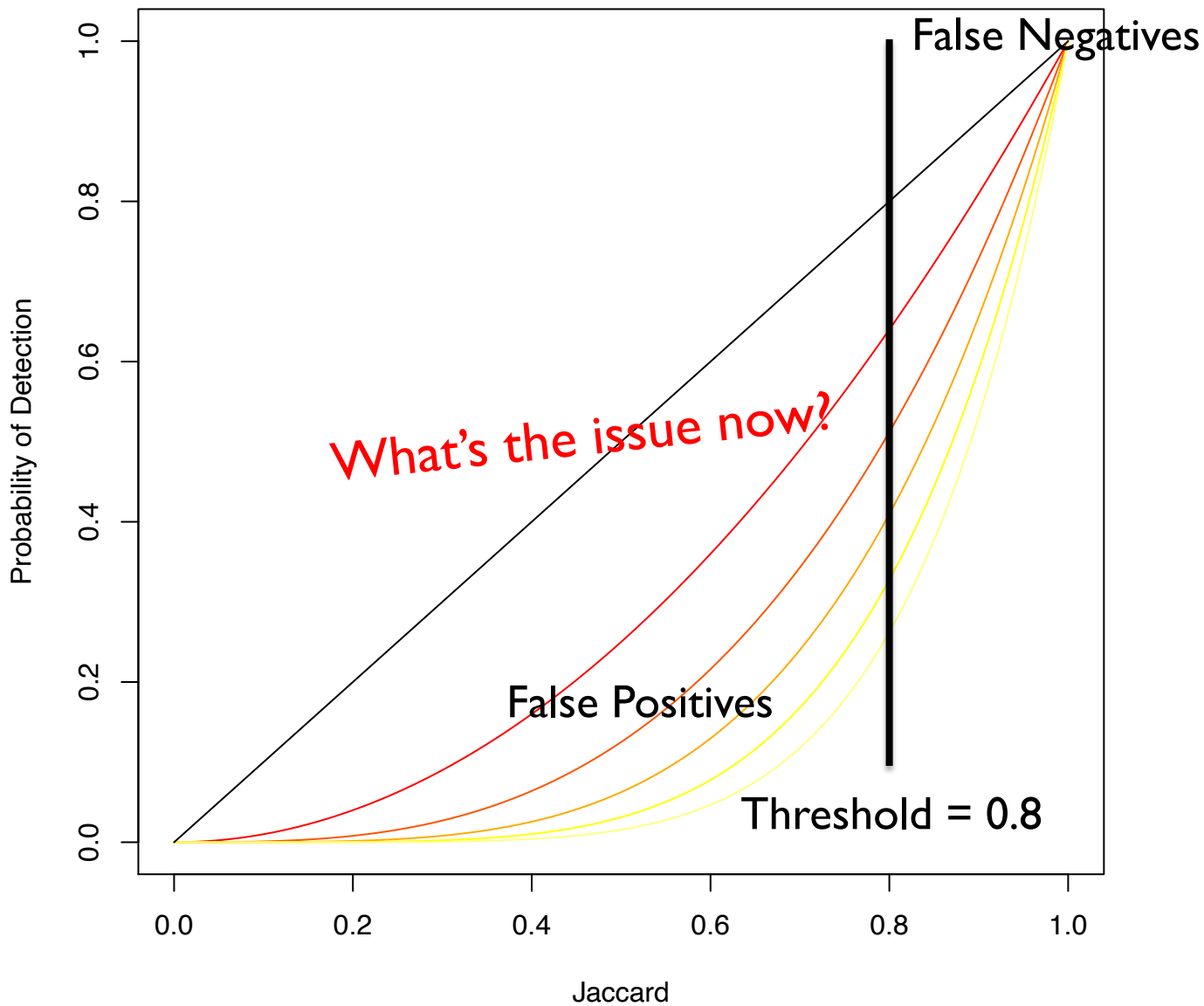
Group objects by their signatures

Output all pairs within each group

Analysis:

If $J(A, B) = s$, then probability we detect it is s^k
(that is, minhash- $\{1 \dots k\}$ all collide)

k Minhash Signatures concatenated together



n different k Minhash Signatures

We know: $P[h(A) = h(B)] = J(A, B)$

Task: discover all pairs with similarity greater than s

Algorithm:

For each object, compute n sets k minhash values

For each set, concatenate k minhash values together

In each set: group objects by signatures, output all pairs in each group

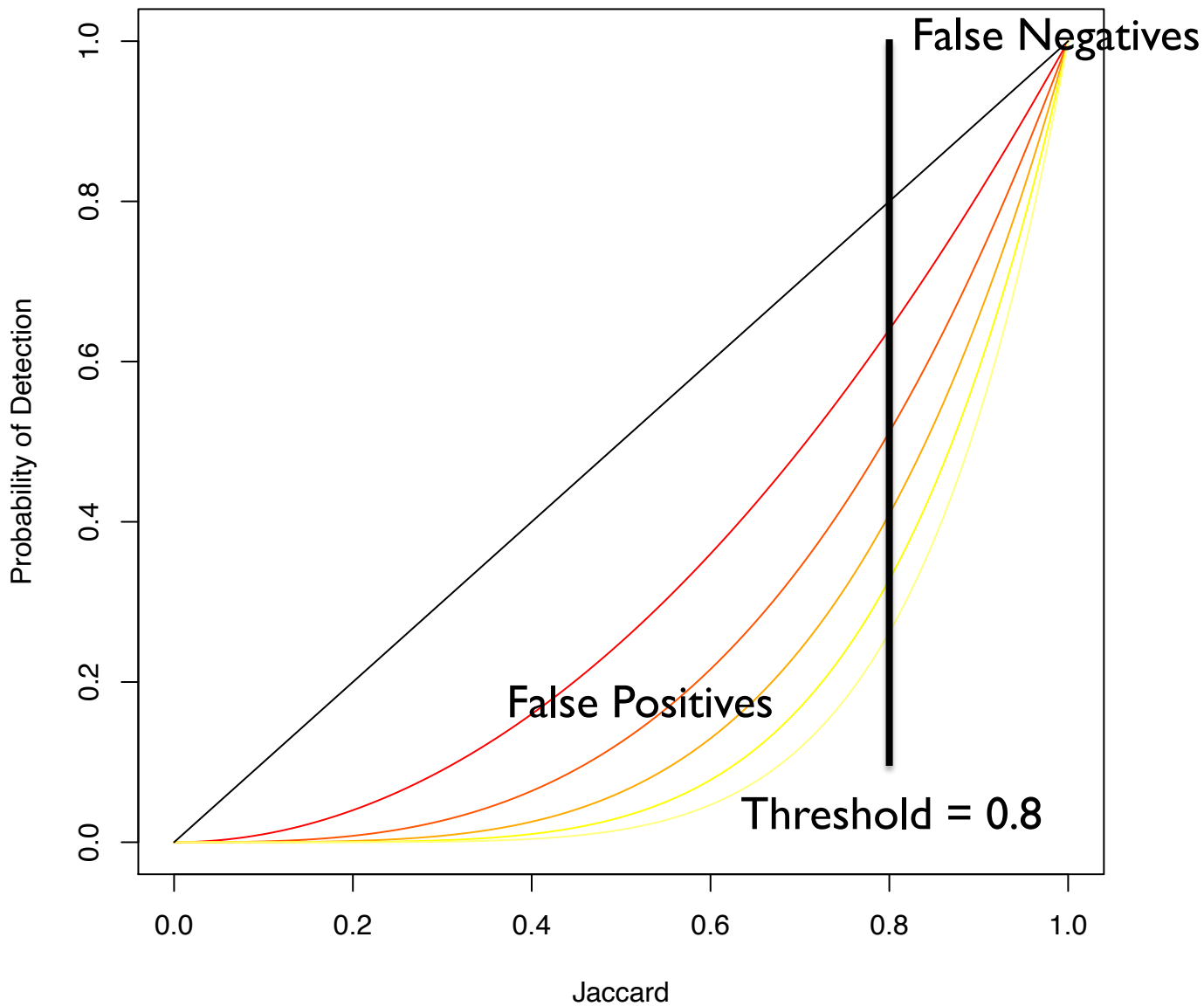
De-dup pairs

Analysis:

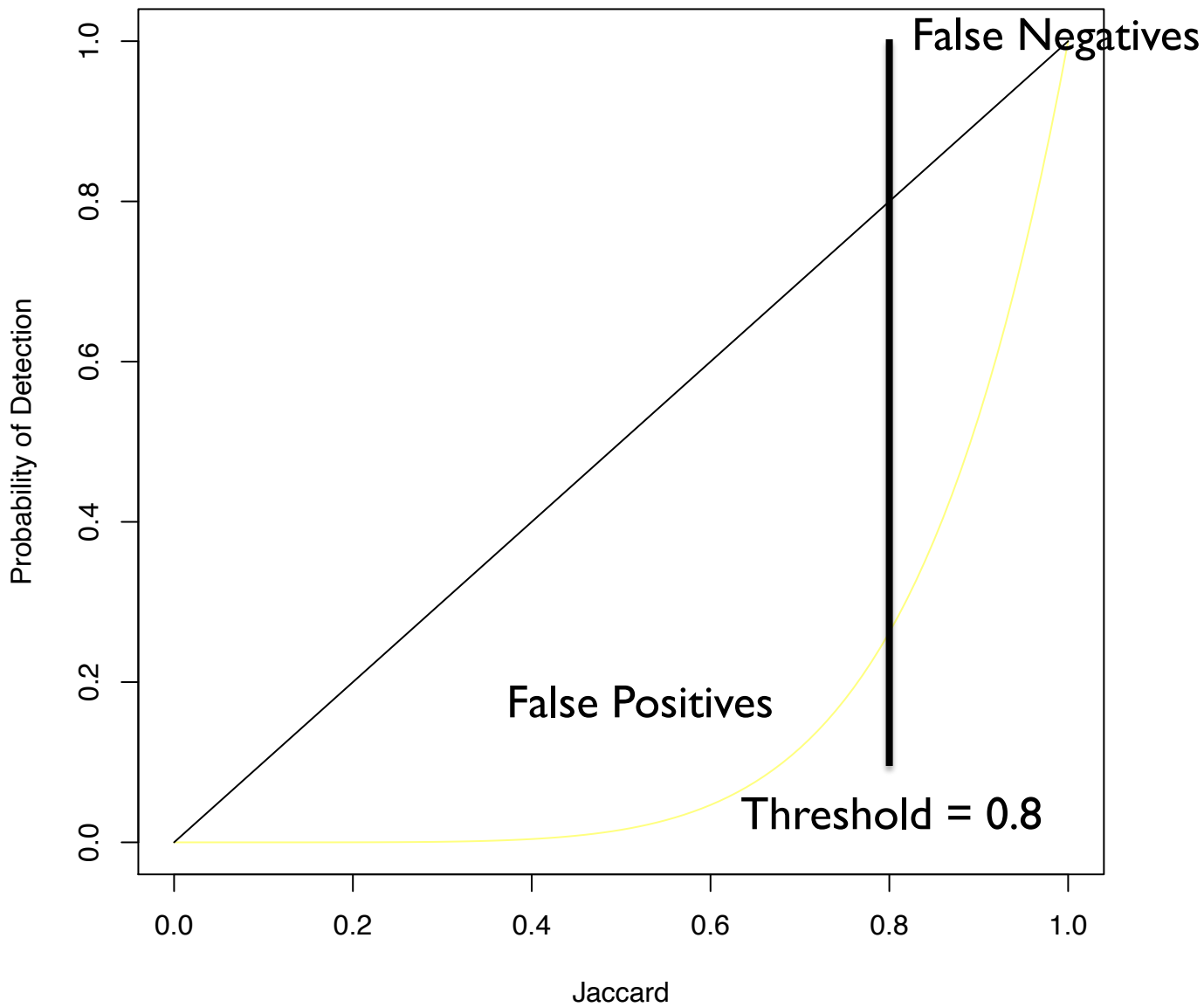
If $J(A, B) = s$, $P(\text{none of the } n \text{ collide}) = (1 - s^k)^n$

If $J(A, B) = s$, then probability we detect it is $1 - (1 - s^k)^n$

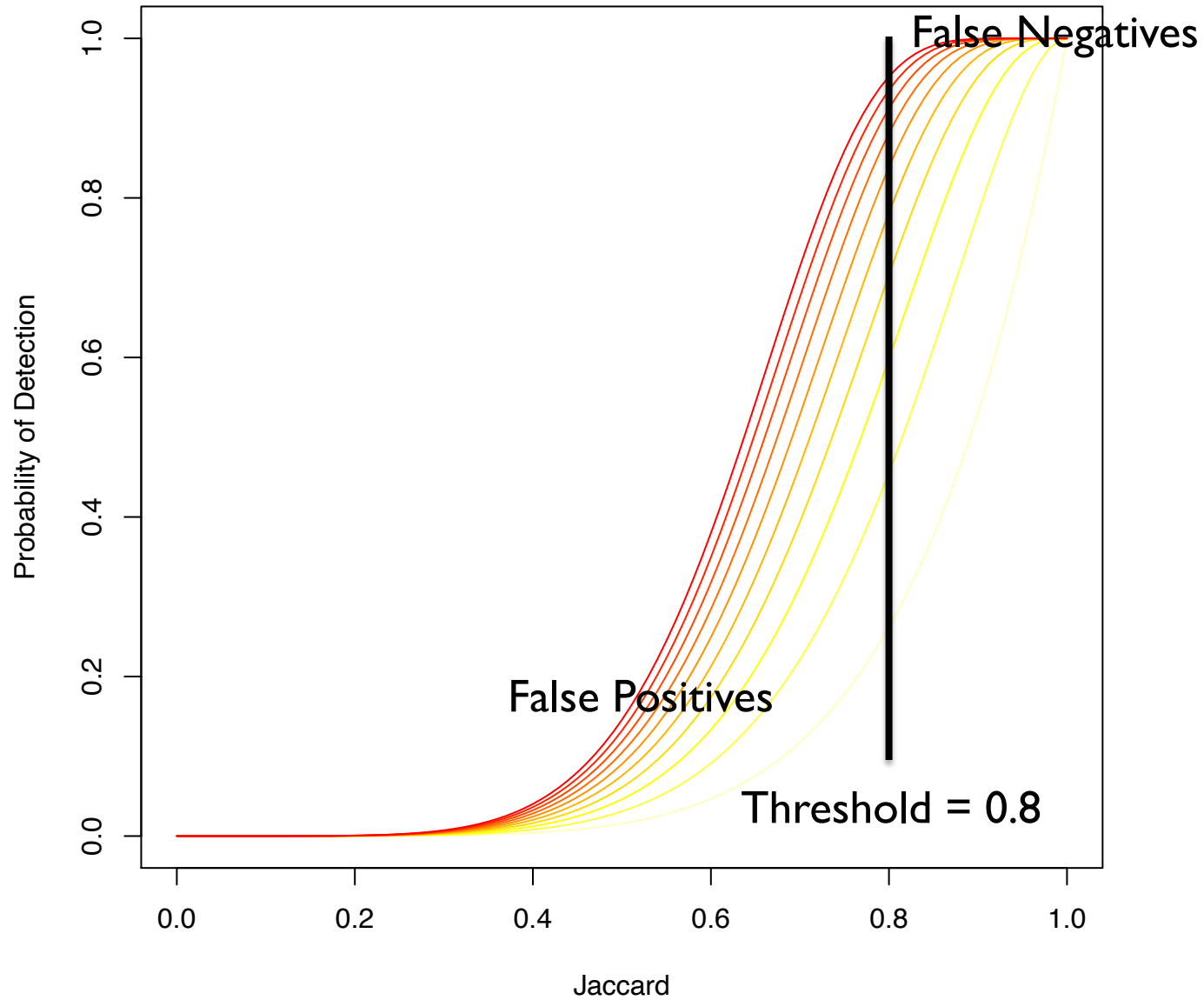
k Minhash Signatures concatenated together



6 Minhash Signatures concatenated together



n different sets of 6 Minhash Signatures



n different k Minhash Signatures

Example: $J(A,B) = 0.8$, 10 sets of 6 minhash signatures

$$P(k \text{ minhash signatures match}) = (0.8)^6 = 0.262$$

$$P(k \text{ minhash signature doesn't match in any of the 10 sets}) = (1 - (0.8)^6)^{10} = 0.0478$$

Thus, we should find $1 - (1 - (0.8)^6)^{10} = 0.952$ of all similar pairs

Example: $J(A,B) = 0.4$, 10 sets of 6 minhash signatures

$$P(k \text{ minhash signatures match}) = (0.4)^6 = 0.0041$$

$$P(k \text{ minhash signature doesn't match in any of the 10 sets}) = (1 - (0.4)^6)^{10} = 0.9598$$

Thus, we should find $1 - (1 - 0.0041)^{10} = 0.040$ of all similar pairs

n different k Minhash Signatures

s	$1 - (1 - s^6)^{10}$
0.2	0.0006
0.3	0.0073
0.4	0.040
0.5	0.146
0.6	0.380
0.7	0.714
0.8	0.952
0.9	0.999

What's the issue?

Practical Notes

Common implementation:

Generate M minhash values, select k of them n times

Reduces amount of hash computations needed

Determining “authoritative” version is non-trivial

MapReduce/Spark Implementation

Map over objects:

Generate M minhash values, select k of them n times

Each draw yields a signature, emit:

key = $(p, \text{signature})$, where $p = [1 \dots n]$ and value = object id

Shuffle/Sort

Reduce

Receive all object ids with same $(n, \text{signature})$, emit clusters

Second pass to de-dup and group clusters

(Optional) Third pass to eliminate false positives

Offline Extraction vs. Online Querying

Batch formulation of the problem:

Discover all pairs with similarity greater than s
Useful for post-hoc batch processing of web crawl

Online formulation of the problem:

Given new webpage, is it similar to one I've seen before?
Useful for incremental web crawl processing

Online Similarity Querying

Preparing the existing collection:

For each object, compute n sets of k minhash values

For each set, concatenate k minhash values together

Keep each signature in hash table (in memory)

Note: can parallelize across multiple machines

Querying and updating:

For new webpage, compute signatures and check for collisions

Collisions imply duplicate (determine which version to keep)

Update hash tables

Random Projections

Limitations of Minhash

Minhash is great for near-duplicate detection

Set high threshold for Jaccard similarity

Limitations:

Jaccard similarity only

Set-based representation, no way to assign weights to features

Random projections:

Works with arbitrary vectors using cosine similarity

Same basic idea, but details differ

Slower but more accurate: no free lunch!

Random Projection Hashing

Generate a random vector r of unit length

Draw from univariate Gaussian for each component

Normalize length

Define:

$$h_r(u) = \begin{cases} 1 & \text{if } r \cdot u \geq 0 \\ 0 & \text{if } r \cdot u < 0 \end{cases}$$

Physical intuition?

RP Hash Collisions

It can be shown that:

Proof in (Goemans and Williamson, 1995)

$$P[h_r(u) = h_r(v)] = 1 - \frac{\theta(u, v)}{\pi}$$

Thus:

$$\cos(\theta(u, v)) = \cos((1 - P[h_r(u) = h_r(v)])\pi)$$

Physical intuition?

Random Projection Signature

Given D random vectors:

$$[r_1, r_2, r_3, \dots, r_D]$$

Convert each object into a D bit signature:

$$u \rightarrow [h_{r_1}(u), h_{r_2}(u), h_{r_3}(u), \dots, h_{r_D}(u)]$$

Since:

$$\cos(\theta(u, v)) = \cos((1 - P[h_r(u) = h_r(v)])\pi)$$

We can derive:

$$\cos(\theta(u, v)) = \cos\left(\frac{\text{hamming}(s_u, s_v)}{D} \cdot \pi\right)$$

Insight: similarity boils down to comparison of hamming distances between signatures

One-RP Signature

Task: discover all pairs with cosine similarity greater than s

Algorithm:

Compute D -bit RP signature for every object

Take first bit, bucket objects into two sets

Perform brute force pairwise (hamming distance) comparison
in each bucket, retain those below hamming distance threshold

Analysis:

Probability we will discover all pairs: *

$$1 - \frac{\cos^{-1}(s)}{\pi}$$

Efficiency

$$N^2 \quad \text{vs.} \quad 2 \left(\frac{N}{2} \right)^2$$

* Note, this is actually a simplification: see Ture et al. (SIGIR 2011) for details.

Two-RP Signature

Task: discover all pairs with cosine similarity greater than s

Algorithm:

Compute D -bit RP signature for every object

Take first two bits, bucket objects into four sets

Perform brute force pairwise (hamming distance) comparison
in each bucket, retain those below hamming distance threshold

Analysis:

Probability we will discover all pairs:

$$\left[1 - \frac{\cos^{-1}(s)}{\pi} \right]^2$$

Efficiency

$$N^2 \quad \text{vs.} \quad 4 \left(\frac{N}{4} \right)^2$$

k -RP Signature

Task: discover all pairs with cosine similarity greater than s

Algorithm:

Compute D -bit RP signature for every object

Take first k bits, bucket objects into 2^k sets

Perform brute force pairwise (hamming distance) comparison
in each bucket, retain those below hamming distance threshold

Analysis:

Probability we will discover all pairs:

$$\left[1 - \frac{\cos^{-1}(s)}{\pi} \right]^k$$

Efficiency

$$N^2 \quad \text{vs.} \quad 2^k \left(\frac{N}{2^k} \right)^2$$

m Sets of k -RP Signature

Task: discover all pairs with cosine similarity greater than s

Algorithm:

Compute D -bit RP signature for every object

Choose m sets of k bits; for each, use k selected bits to bucket objects into 2^k sets

Perform brute force pairwise (hamming distance) comparison
in each bucket, retain those below hamming distance threshold

Analysis:

Probability we will discover all pairs:

$$1 - \left[1 - \left[1 - \frac{\cos^{-1}(s)}{\pi} \right]^k \right]^m$$

Efficiency

$$N^2 \quad \text{vs.} \quad m \cdot 2^k \left(\frac{N}{2^k} \right)^2$$

MapReduce/Spark Implementation

Map over objects:

Compute D -bit RP signature for every object

Choose m sets of k bits and use to bucket; for each, emit:

key = $(p, k \text{ bits})$, where $p = [1 \dots m]$, value = (object id, rest of signature bits)

Shuffle/Sort

Reduce

Receive $(p, k \text{ bits})$

Perform brute force pairwise (hamming distance) comparison for each key,
retain those below hamming distance threshold

Second pass to de-dup and group clusters

(Optional) Third pass to eliminate false positives

Online Querying

Preparing the existing collection:

Compute D -bit RP signature for every object

Choose m sets of k bits and use to bucket

Store signatures in memory (across multiple machines)

Querying:

Compute D -bit signature of query object, choose m sets of k bits in same way

Perform brute-force scan of correct bucket (in parallel)

Additional Issues to Consider

Emphasis on recall, not precision

Two sources of error:

From LSH

From using hamming distance as proxy for cosine similarity

Load imbalance

Parameter tuning

LSH: Four Steps

Specify distance metric
Jaccard, Euclidean, cosine, etc.

Compute representation
Shingling, tf.idf, etc.

Hash
Minhash, random projections, etc.

Examine collisions
Process items in same “bucket”

富嶽三十六景 神奈川
浪裏

